

Generative AI Models (OpenRouter – Latest 12 Months)

1) Find 10 different model providers and their models released in last 12 months

1. OpenAI — GPT-5.5

- Release Date: April 24, 2026
- Cutoff Date: December 1, 2025
- Cost: \$5/M input, \$30/M output
- Context Window: 1M tokens
- Public Model: Yes
- When to Use: reasoning, coding, AI agents, multimodal tasks

2. Anthropic — Claude Sonnet 4.6

- Release Date: February 17, 2026
- Cutoff Date: May 2025 (not officially fixed on OpenRouter)
- Cost: \$3/M input, \$15/M output
- Context Window: 1M tokens
- Public Model: Yes
- When to Use: reasoning, coding, long-context tasks, enterprise AI

3. Google — Gemini 3 Flash Preview

- Release Date: December 17, 2025
- Cutoff Date: January 2025
- Cost: \$0.50/M input, \$3.00/M output
- Context Window: 1M tokens
- Public Model: Yes
- When to Use: fast AI, coding, multimodal tasks, long-context reasoning

4.4. Meta — Llama 4 Maverick

- Release Date: April 5, 2025
- Cutoff Date: August 31, 2024
- Cost of Tokens: \$0.15/M input, \$0.60/M output
- Context Window: 1M tokens
- Public Model: Yes
- When to Use: reasoning, coding, multimodal AI, long-context tasks

5. xAI — Grok-4.20

- Release Date: March 31, 2026
- Cutoff Date: September 1, 2025

- Cost: \$2/M input, \$6/M output
- Context Window: 2M tokens
- Public Model: Yes
- When to Use: reasoning, coding, AI agents, long-context tasks

6. DeepSeek — DeepSeek R1

- Release Date: January 20, 2025
- Cutoff Date: July 31, 2024
- Cost: \$0.70/M input, \$2.50/M output
- Context Window: 64K tokens
- Public Model: Yes
- When to Use: reasoning, coding, math, logical problem solving

7. Mistral AI — Mistral Medium 3

- Release Date: May 7, 2025
- Cutoff Date: March 31, 2025
- Cost of Tokens: \$0.40/M input, \$2.00/M output
- Context Window: 131K tokens
- Public Model: Yes
- When to Use: reasoning, coding, multimodal tasks, enterprise AI

8. Cohere — Command A

- Release Date: March 13, 2025
- Cutoff Date: August 31, 2024
- Cost: \$2.50/M input, \$10/M output
- Context Window: 256K tokens
- Public Model: Yes
- When to Use: enterprise AI, RAG systems, multilingual tasks, agent workflows

9. Qwen — Qwen3.6 Plus

- Release Date: April 2, 2026
- Cutoff Date: Not publicly specified
- Cost: \$0.325/M input, \$1.95/M output
- Context Window: 1M tokens
- Public Model: Yes
- When to Use: coding, reasoning, multimodal AI, long-context tasks

10. NVIDIA — Llama 3.3 Nemotron Super 49B v1.5

- Release Date: March 18, 2025
- Cutoff Date: December 31, 2023
- Cost: \$0.10/M input, \$0.40/M output
- Context Window: 131K tokens
- Public Model: Yes
- When to Use: reasoning, coding, AI agents, long-context tasks

2)Transformer in Gen AI:

Referred documents:

1.<https://jalammar.github.io/illustrated-transformer/>

2.<https://www.geeksforgeeks.org/deep-learning/architecture-and-working-of-transformers-in-deep-learning/>

A Transformer model works by converting words into vector embeddings, using self-attention to understand relationships between words, and passing results through feed-forward neural networks to learn complex patterns. This allows it to process text efficiently and understand long-range dependencies.

Transformer flow:

Step-by-step process:

1. Input text → Tokenization
2. Embedding layer → converts words to vectors
3. Self-Attention → finds relationships between words
4. Feed Forward Network → processes each token
5. Repeat layers multiple times
6. Output prediction (next word / answer)

Architecture of Transformer

The Transformer has two main parts:

1. Encoder

- Takes input sequence and converts it into meaningful representations
- Each encoder layer has:
 - Self-Attention layer
 - Feed Forward Neural Network (FFN)

2. Decoder

- Generates output sequence using encoder output
- Each decoder layer includes:
 - Masked Self-Attention
 - Encoder-Decoder Attention
 - Feed Forward Network

MAIN COMPONENTS OF TRANSFORMERS:

1.Vector Embeddings

- Words are converted into numbers (vectors)
- Example:
"king" → [0.2, -1.5, 3.0, ...]
- These vectors capture meaning of words
- Similar words have similar vectors

2. Self-Attention Mechanism

- Core idea of Transformer
- Each word looks at all other words in a sentence
- It decides which words are important

♦ Uses 3 vectors:

- Query (Q) → What I am looking for
- Key (K) → What I contain
- Value (V) → Actual information

♦ Working:

- Compare Q with K → find importance score
- Multiply score with V → final output

Helps understand context like:

- "It" refers to correct object in sentence

3. Feed Forward Network (FFN)

- After attention, data goes to a small neural network
- Applied independently to each word

Structure:

- Linear layer → Activation (ReLU/GELU) → Linear layer

Purpose:

- Adds non-linearity
- Learns complex patterns
- Improves representation of words